UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación Paralela y Distribuida

Transformada lineal de Hough en CUDA

Comparación de memoria global, constante y compartida

Integrantes (autores del repositorio)
Ihan-Marroquin
dpatzan2
G1LB3T0

Docente: Sebastián Galindo
Proyecto 3
Fecha de entrega: 11 de noviembre de 2026

Mediciones: 6 de noviembre de 2026, 18:21, Guatemala (UTC-6).

Repositorio: UVG-Tareas-y-proyectos / PRY3_CUDA

1. Índice e introducción

Contenido	Página
1. Índice e introducción	2
2. Algoritmo y coordenadas	3
3. Memoria global y constante	4
4. Memoria compartida y sincronización	5
5. Metodología y recursos	6
6. Resultados y análisis	7
7. Visualización de líneas	8
8. Bitácora de mediciones	9
9. Validación y conclusiones	10
10. Referencias	11
Apéndice. Reproducción y exposición	12

El objetivo del proyecto es detectar rectas en una imagen de bordes y comparar tres formas de distribuir los datos de la Transformada de Hough en CUDA. El trabajo conserva la discretización del código del curso: 90 ángulos, separados por 2 grados, y 100 intervalos de distancia. Cada píxel no nulo emite un voto por ángulo.

La implementación entrega una imagen PNG con las rectas superpuestas a color, un CSV con sus parámetros y una bitácora de 30 tiempos por variante. La versión CPU sirve como referencia exacta y cada ejecución medida comprueba la igualdad de los 9000 contadores.

En esta sesión, la media del kernel compartido fue 0.726596 ms frente a 1.148245 ms del global. La aceleración relativa fue 1.58x. Estos resultados corresponden al tiempo del kernel en la RTX 3060, con las condiciones descritas en la metodología.

2. Algoritmo y coordenadas

Una recta se representa por su distancia firmada rho respecto del origen y por el ángulo theta de su normal. Para un punto cartesiano (x, y), todos los parámetros que satisfacen rho = x cos(theta) + y sin(theta) describen rectas que pasan por ese punto. La transformación discretiza esa familia y acumula votos. Los máximos identifican familias de puntos alineados. [1, 5]

Discretización del proyecto

Parámetro	Valor
theta	0, 2, 4, ..., 178 grados (90 bins)
rMax	sqrt(ancho² + alto²) / 2 = 500 píxeles
rScale	2 rMax / 100 = 10 píxeles
Acumulador	100 x 90 = 9000 enteros (36000 bytes)

El índice de un píxel en memoria es id = fila * ancho + columna. El kernel recupera la columna mediante id % ancho y la fila mediante id / ancho, con división entera. Entonces calcula xCoord = columna - ancho / 2 y yCoord = alto / 2 - fila. El desplazamiento coloca el origen en el centro y el cambio de signo hace que el eje y crezca hacia arriba.

Por ejemplo, para 800 x 600, el píxel de columna 400 y fila 300 pasa a (0, 0); el de columna 400 y fila 299 pasa a (0, 1). Esta convención se usa tanto al votar como al dibujar las líneas, para evitar inversiones de orientación.

Cada voto obtiene rIndex = int((rho + rMax) / rScale). El desplazamiento convierte las distancias negativas en índices no negativos. Un límite explícito al intervalo [0, 99] protege los extremos frente al redondeo. El índice lineal del acumulador es rIndex * 90 + thetaIndex.

Paralelismo y costo

La geometría usa bloques unidimensionales de 256 hilos y un hilo por píxel. El ID global es blockIdx.x * blockDim.x + threadIdx.x; el número de bloques es (N - 1) / 256 + 1. Para runway, N = 480000 y se lanzan 1875 bloques. El trabajo principal es proporcional a 90 veces el número de píxeles activos: 12456540 votos en esta imagen.

Aunque cada hilo procesa un píxel distinto, varios píxeles pueden pertenecer a una misma recta y actualizar el mismo contador. Una suma ordinaria perdería incrementos concurrentes. Por ello las actualizaciones GPU emplean atomicAdd. La CPU recorre los píxeles secuencialmente y no requiere atómicos.

3. Memoria global y constante

La versión global almacena imagen, acumulador y tablas trigonométricas en memoria global. El host calcula una sola vez los 90 senos y 90 cosenos. Cada hilo consulta las tablas dentro del bucle angular y vota directamente en el acumulador global mediante atomicAdd.

En la segunda versión, los arreglos constantCos y constantSin se declaran con __constant__ fuera de main. El host transfiere los datos con cudaMemcpyToSymbol y el kernel deja de recibir punteros a las tablas. La imagen y el acumulador permanecen en memoria global. Las dos tablas ocupan 720 bytes y no cambian durante la ejecución. [2]

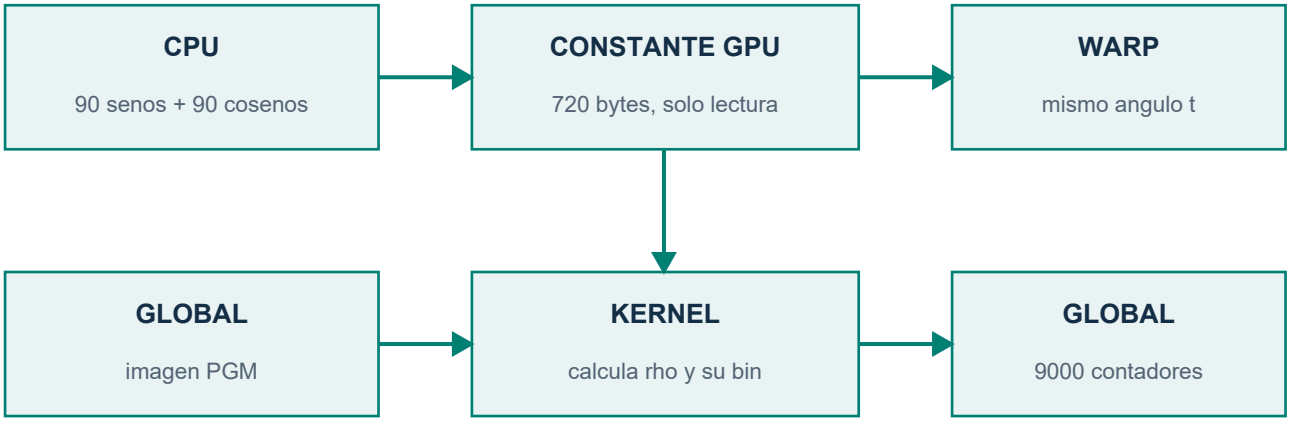


Figura 1. Flujo funcional de la variante global + constante. Elaboración propia a partir de la implementación y del modelo de memoria CUDA. [2]

La caché constante favorece accesos uniformes: los hilos activos de un warp solicitan el mismo índice t durante cada iteración. Esta propiedad permite difundir un valor a múltiples hilos. Si los hilos solicitan direcciones distintas, el beneficio puede disminuir. La capacidad de la memoria y su caché dependen del dispositivo; no se debe asumir que toda lectura constante siempre será más rápida. [2, 3]

En las 30 mediciones, la variante constante tardó 1.328060 ms de media, un 15.66% más que la global. Por tanto, mover únicamente las tablas no produjo una mejora en esta sesión. El tráfico de las tablas es pequeño frente a los votos atómicos, y la versión global también puede beneficiarse de cachés. Son explicaciones posibles: no se midieron transacciones ni ocupación para atribuir una causa única.

La documentación moderna de NVIDIA se utiliza para interpretar los resultados. Las afirmaciones de capacidad o latencia de una GPU antigua no se trasladan automáticamente a la RTX 3060 utilizada en este experimento. [2, 3]

4. Memoria compartida y sincronización

La tercera versión mantiene las tablas en constante y crea __shared__ int localAcc[9000] dentro del kernel. Cada bloque posee su propio acumulador de 36000 bytes. La inicialización se reparte entre los hilos: cada locID = threadIdx.x pone a cero las posiciones locID, locID + blockDim.x y así sucesivamente. Una primera barrera garantiza que todos los contadores estén listos antes de votar. [2, 4]



Cada bloque posee su propio localAcc: 9000 x 4 = 36000 bytes.

Figura 2. Acumulación cooperativa dentro de cada bloque y fusión final en memoria global. Elaboración propia. [2, 4]

Cada píxel activo vota con atomicAdd sobre localAcc. Los atómicos siguen siendo necesarios porque varios hilos del mismo bloque pueden elegir el mismo bin. Después, una segunda barrera evita que la fusión lea votos incompletos. Finalmente, los hilos recorren el acumulador con el mismo patrón por saltos y suman sus posiciones no nulas al acumulador global, también con atomicAdd, porque otros bloques fusionan simultáneamente.

Los hilos que quedan fuera de la imagen en un bloque parcial no leen píxeles ni votan, pero sí inicializan, sincronizan y fusionan. Un return antes de cualquiera de las barreras invalidaría la cooperación del bloque. Por eso las barreras se encuentran fuera de la condición globalId < ancho * alto. [4]

La media observada fue 0.726596 ms, equivalente a 1.58x respecto del global. La agrupación local puede reducir contención global al combinar votos antes de fusionarlos. Su costo incluye poner a cero 9000 enteros por bloque, dos barreras y la fusión. Además, 36 kB por bloque consumen un recurso que limita la concurrencia. El balance depende de la imagen y del equipo; la mejora observada no demuestra un aumento de ocupación. [3]

5. Metodología y recursos

Elemento	Configuración registrada
GPU	NVIDIA GeForce RTX 3060 Laptop GPU
Driver / CUDA	592.82 / CUDA 13.1, nvcc 13.1.115
Host	Ubuntu 26.04 en WSL2; GCC 14.3; Windows WDDM
Compilación	-O2, C++17, sm_86, --fmad=false, -ffp-contract=off
Datos	800 x 600, 138406 píxeles no nulos
Repeticiones	5 calentamientos y 30 mediciones por variante
Memorias	720 B constante; 36000 B compartida por bloque

Se crea un par de CUDA events por variante. Cada repetición pone el acumulador en cero, registra el evento inicial, lanza el kernel, registra el evento final, espera ese evento y consulta cudaEventElapsedTime. El reinicio queda antes del evento inicial. La copia a CPU, la comparación y la escritura de archivos ocurren después del evento final. Así se mide ejecución del kernel, incluyendo su sincronización interna, en milisegundos. [1]

Después de cada repetición medida se copian los 9000 bins y se exige igualdad exacta con la referencia CPU. Una discrepancia produce código de salida distinto de cero y evita publicar resultados aceptados. Todos los kernels y la CPU usan las mismas tablas float. La compilación desactiva la fusión multiplicación-suma para reducir diferencias de redondeo al discretizar rho.

Los datos se recolectaron el 6 de noviembre de 2026 a las 18:21 en Guatemala. El orden fue global, constante y compartida. La GPU también atiende el escritorio; no se fijaron relojes ni se aisló térmicamente el equipo. Por ello se publican todos los valores, media, mediana, mínimo, máximo y desviación estándar muestral. La comparación es descriptiva para esta sesión.

El uso de vectores en CPU, buffers de dispositivo y eventos con RAII libera los recursos al terminar su ámbito y también durante excepciones. Después de destruir esos recursos, main llama cudaDeviceReset. Se comprueban los estados de reserva, copia, eventos y lanzamiento. La lectura PGM rechaza datos incompletos, valores fuera de rango y dimensiones incompatibles con los índices de 32 bits.

La instalación aislada usa cabeceras glibc 2.39 para compilar con CUDA 13.1 en Ubuntu 26.04, cuyo sistema tiene glibc 2.43. Esto resuelve el conflicto de declaraciones rsqrt/rsqrtf sin alterar las bibliotecas del sistema. scripts/setup_toolchain.py reproduce esa preparación desde descargas oficiales verificadas por SHA256.

6. Resultados y análisis

Variante	Media ms	Mediana ms	Desv. ms	Rango ms
Global	1.148245	1.119232	0.135710	1.000 - 1.434
Constante	1.328060	1.312768	0.038624	1.288 - 1.410
Compartida	0.726596	0.726016	0.030939	0.687 - 0.781

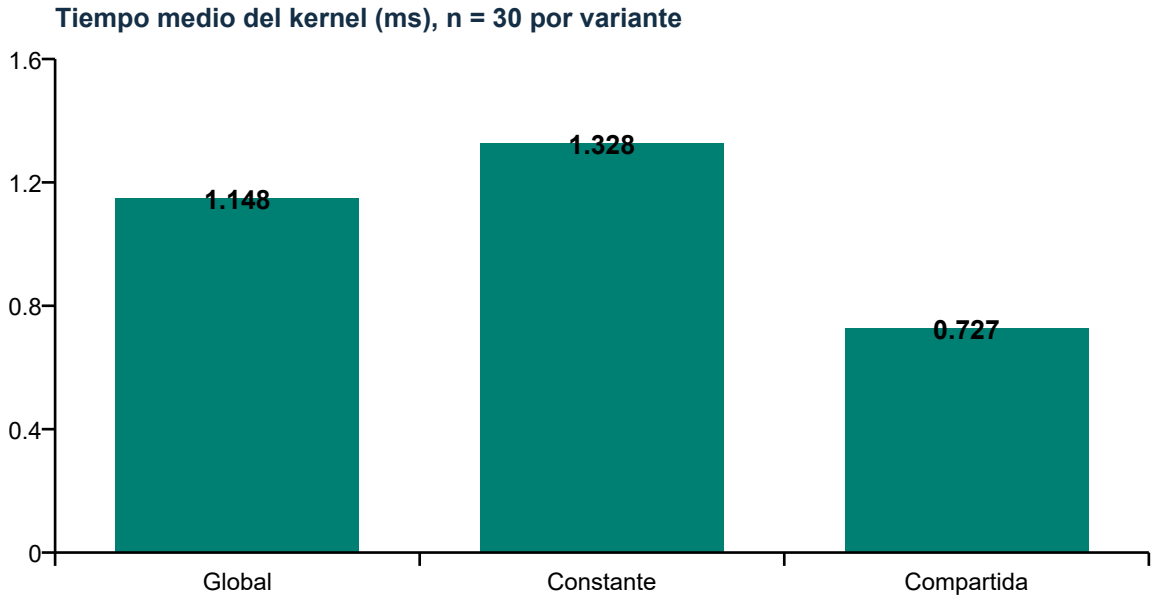


Figura 3. Comparación descriptiva de medias. Fuente: results/runway_timings.csv. La barra más baja corresponde a menor tiempo. Los tiempos individuales están en la página 9.

El cociente media global / media compartida es 1.5803. La disminución relativa del tiempo es 1 - 0.726596 / 1.148245 = 36.72%. La versión constante obtiene un cociente de 0.8646, lo que corresponde a 15.66% más tiempo que la global. Estos cocientes comparan kernels que producen el mismo acumulador.

La mayor dispersión relativa corresponde a la versión global, con desviación muestral de 0.135710 ms. La mediana ayuda a describir su centro sin depender tanto de las mediciones extremas. La dispersión también muestra por qué una sola ejecución no basta para establecer un resultado de rendimiento.

La referencia CPU tardó 26.825217 ms en una única ejecución. Ese dato sirve de contexto y de validación, pero no es una serie estadística comparable a las 30 mediciones GPU. Tampoco representa un speedup de toda la aplicación, porque el intervalo GPU excluye reservas, transferencias y generación del PNG.

7. Visualización de líneas

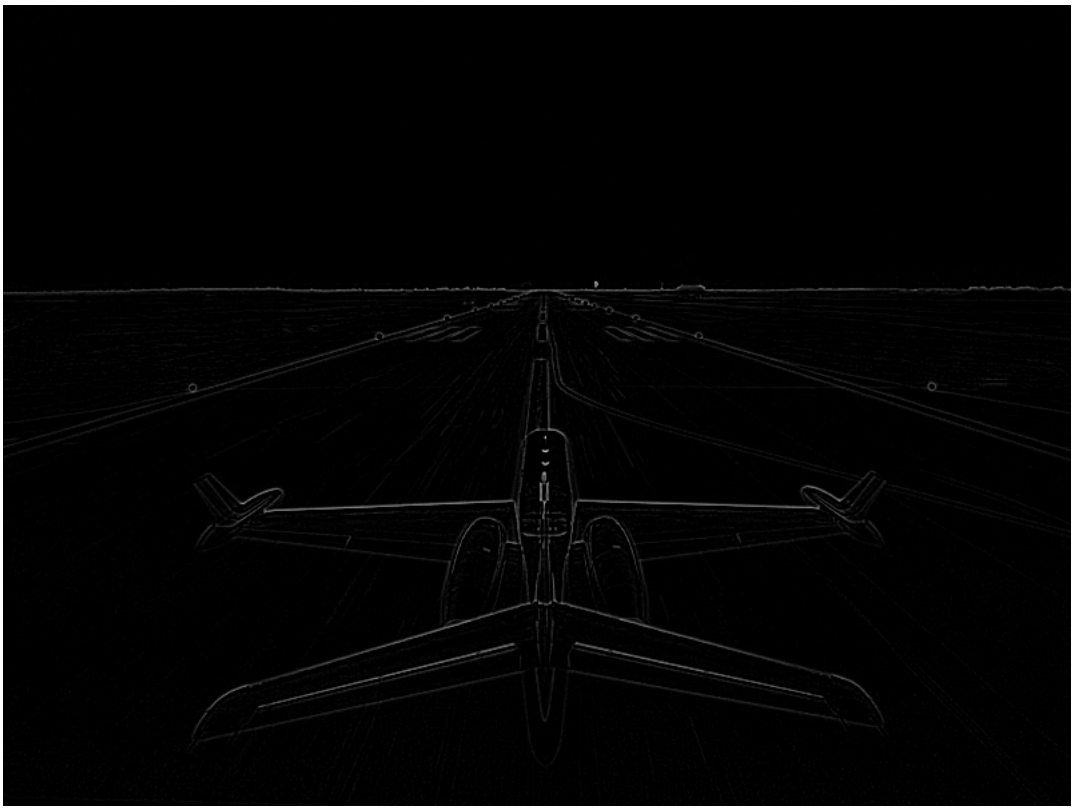


Figura 4. Imagen original de bordes, conservada sin modificaciones. Fuente: archivo runway.pgm proporcionado por el curso.

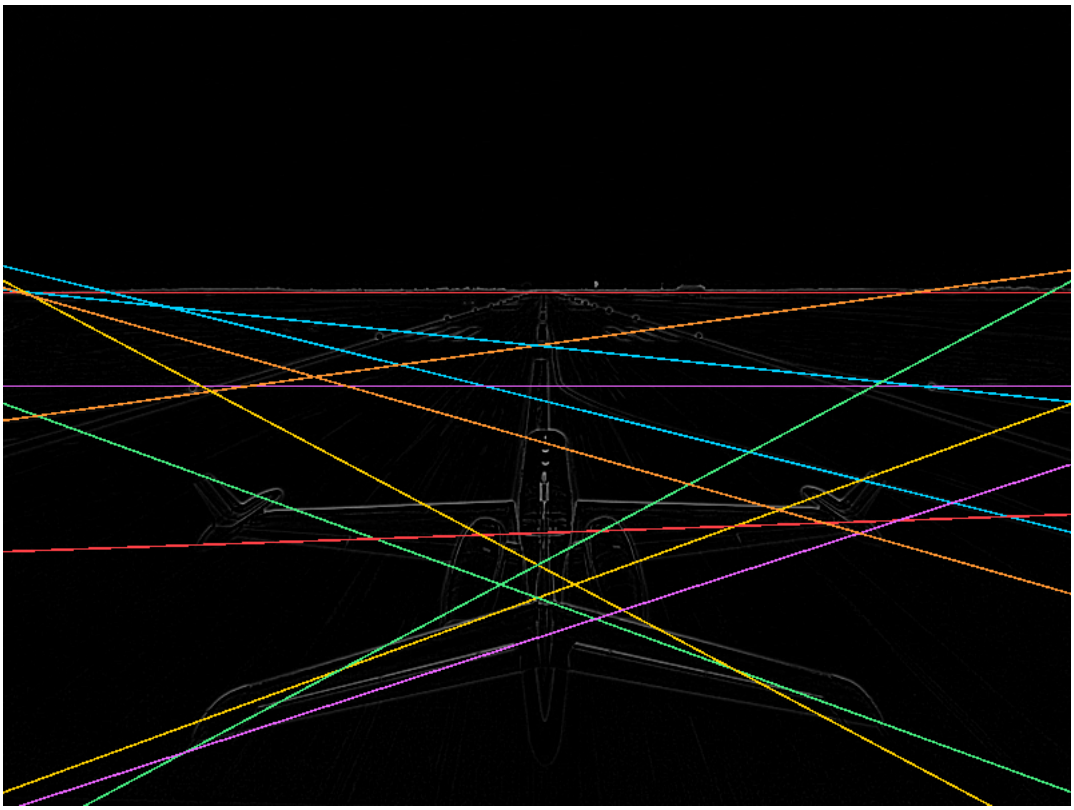


Figura 5. Salida del programa. Doce rectas sobre la imagen original; votos > 2772. El color distingue las rectas, sin codificar confianza ni clase semántica.

8. Bitácora de mediciones

Treinta mediciones por variante mediante CUDA events. Todas las celdas contienen milisegundos. Las 90 ejecuciones medidas coincidieron exactamente con la CPU; los cinco calentamientos de cada variante no están en esta tabla.

Repetición	Global (ms)	Constante (ms)	Compartida (ms)
1	1.020928	1.401856	0.726016
2	1.009664	1.330176	0.724992
3	1.039360	1.393664	0.772096
4	1.001472	1.375232	0.768000
5	1.018880	1.346560	0.781312
6	1.030144	1.390592	0.729088
7	1.000448	1.410048	0.724992
8	1.005568	1.358848	0.723968
9	1.003520	1.295360	0.768000
10	1.433600	1.299456	0.758784
11	1.043456	1.302528	0.726016
12	1.004544	1.295360	0.743424
13	1.044480	1.315840	0.762880
14	1.008640	1.309696	0.755712
15	1.013760	1.337344	0.756736
16	1.069056	1.345536	0.720896
17	1.169408	1.336320	0.744448
18	1.315840	1.303552	0.750592
19	1.290240	1.347584	0.693248
20	1.266688	1.342464	0.734208
21	1.257472	1.373184	0.688128
22	1.260544	1.296384	0.690176
23	1.255424	1.290240	0.687104
24	1.250304	1.290240	0.687104
25	1.256448	1.288192	0.731136
26	1.258496	1.291264	0.692224
27	1.248256	1.291264	0.688128
28	1.302528	1.292288	0.688128
29	1.288192	1.295360	0.687104
30	1.280000	1.295360	0.693248

Fuente: results/runway_timings.csv. La desviación estándar publicada usa divisor n - 1. scripts/benchmark.py calcula el resumen directamente de estos valores y registra las huellas SHA256 del binario y de la imagen.

9. Validación y conclusiones

Validación funcional

La batería automática aprobó 23 casos. Cubre imagen negra, un solo píxel, 257 píxeles (bloque final parcial), imagen densa de dimensiones impares, puntos aleatorios, líneas verticales, horizontales y diagonales, y bytes que podrían confundirse con espacios en PGM binario. También rechaza cabeceras vacías, formatos no admitidos, dimensiones cero o negativas, datos truncados, profundidades de 16 bits y valores fuera de rango.

Las pruebas geométricas comprueban picos esperados para líneas horizontales y verticales y el efecto de discretización sobre una diagonal de 45 grados. Además, verifican dimensiones, canal RGB, CRC de los chunks PNG y descompresión zlib. La referencia CPU comprueba conservación de votos: 90 por cada píxel activo.

Compute Sanitizer intentó memcheck, racecheck y synccheck, pero WDDM rechazó la interfaz de depuración. Por ello no se afirma que la instrumentación haya demostrado ausencia de carreras o fugas. Los logs conservan el fallo y make sanitize permite repetir las verificaciones en un entorno compatible. La igualdad CPU/GPU y las pruebas funcionales sí se ejecutaron en hardware real.

Interpretación visual y límites

La selección ordena los bins por votos y acepta únicamente pesos estrictamente superiores al umbral. Suprime candidatos a una distancia de hasta 2 bins radiales y 4 bins angulares de una recta aceptada, incluida la equivalencia de ángulos cerca de 0/180 grados. Se limita la salida a 12 rectas. Este procedimiento reduce duplicados, pero no clasifica las rectas como bordes de pista ni distingue objetos.

El dibujo usa el centro de cada bin radial. Con pasos de 10 píxeles puede existir un desplazamiento de hasta aproximadamente medio bin, además del error angular. Cada recta se prolonga hasta los límites de la imagen; el algoritmo no estima segmentos ni extremos. Un umbral configurable permite reducir rectas secundarias o aumentar sensibilidad.

Conclusiones y recomendaciones

La implementación satisface el cálculo del ID global, la comparación exacta de resultados y la salida visual. Las barreras fuera de la condición de píxel válido permiten procesar bloques incompletos correctamente. Los atómicos son necesarios tanto al votar dentro del bloque como al fusionar entre bloques.

En esta sesión, la variante compartida redujo el tiempo medio del kernel un 36.72% frente a la global. La constante sola no mejoró el rendimiento. Para generalizar el resultado conviene repetir las series con orden aleatorio, controlar el estado térmico y medir contención y ocupación con un perfilador. Cambiar resolución de bins o tamaño de bloque requiere volver a comprobar tanto exactitud como uso de memoria.

10. Referencias

[1] Harris, M. (2012, 7 de noviembre). How to Implement Performance Metrics in CUDA C/C++. NVIDIA Technical Blog.

Consultar fuente oficial de NVIDIA

[2] NVIDIA. (2025). CUDA C++ Programming Guide, versión 12.9.1. Jerarquía de memoria, calificadores __constant__/__shared__, atómicos y sincronización.

Consultar fuente oficial de NVIDIA

[3] NVIDIA. (2025). CUDA C++ Best Practices Guide, versión 12.9.1. Medición, memoria, cachés y ocupación.

Consultar fuente oficial de NVIDIA

[4] Harris, M. (2013, 28 de enero). Using Shared Memory in CUDA C/C++. NVIDIA Technical Blog.

Consultar fuente oficial de NVIDIA

[5] Galindo, S. (s. f.). Proyecto 3 - CUDA. Universidad del Valle de Guatemala. Guía de proyecto y rúbrica proporcionadas por el curso. Se utiliza la fecha de entrega indicada por el equipo: 11 de noviembre de 2026.

[6] Barlas, G. (2014, diciembre). houghBase.cu, versión 1.0. Código fuente entregado con el proyecto; cabecera con licencia GNU GPL 3.0. La atribución se conserva en NOTICE.md y en el código.

Los números entre corchetes remiten a esta bibliografía. Las descripciones de implementación y las figuras funcionales documentan el código del repositorio. Los resultados cuantitativos proceden exclusivamente de los archivos de la ejecución final, sin extrapolar rendimientos de otros equipos.

Material suplementario conservado

El repositorio incluye el código CUDA y C++, la imagen original, el Makefile, pruebas automáticas, scripts de compilación y medición, las 90 mediciones sin resumir, metadatos del entorno, la salida PNG, los parámetros de las líneas y los logs de validación. El informe y la presentación se entregan junto con el código, conforme a la indicación de no enviar solamente un enlace.

Apéndice. Reproducción y exposición

Compilación con CUDA instalado

```
make

make test

make benchmark RUNS=30

make sanitize
```

El valor predeterminado CUDA_ARCH=86 corresponde a la RTX 3060. Puede ajustarse a otra arquitectura soportada por el toolkit. HOST_CXX y EXTRA_NVCCFLAGS permiten seleccionar el compilador y sus rutas sin modificar el código.

Preparación aislada para esta instalación WSL2

```
python3 scripts/setup_toolchain.py

source .toolchain/env.sh

make

make test
```

La preparación requiere make y g++-14. Descarga componentes oficiales CUDA 13.1 y cabeceras Ubuntu a .toolchain/, directorio ignorado por Git. No instala un driver Linux: WSL usa el driver NVIDIA de Windows.

Ejecución y ajuste del umbral

```
./build/hough data/runway.pgm --runs 30 \
--threshold 4000 --max-lines 8 --output results/custom
```

La opción --threshold compara con >, por lo que un peso exactamente igual al umbral no se dibuja. El prefijo determina los cuatro archivos de salida; el script benchmark agrega un resumen y el registro de ejecución. Para regenerar este informe: pip install reportlab pillow y python scripts/build_report.py.

Organización de la exposición oral

Participación sugerida	Contenido
dpatzan2	Algoritmo, coordenadas y sincronización del bloque
G1LB3T0	Salida PNG, umbral, memorias y demostración
Ihan-Marroquin	CUDA events, bitácora, resultados y límites

La presentación ejecutiva incluye notas para apoyar la explicación. La demostración puede ejecutar make run y abrir results/runway_lines.png. Cada integrante debe poder explicar por qué varios píxeles compiten por un mismo bin y por qué todos los hilos alcanzan las barreras. La preparación y participación oral corresponden al equipo.